

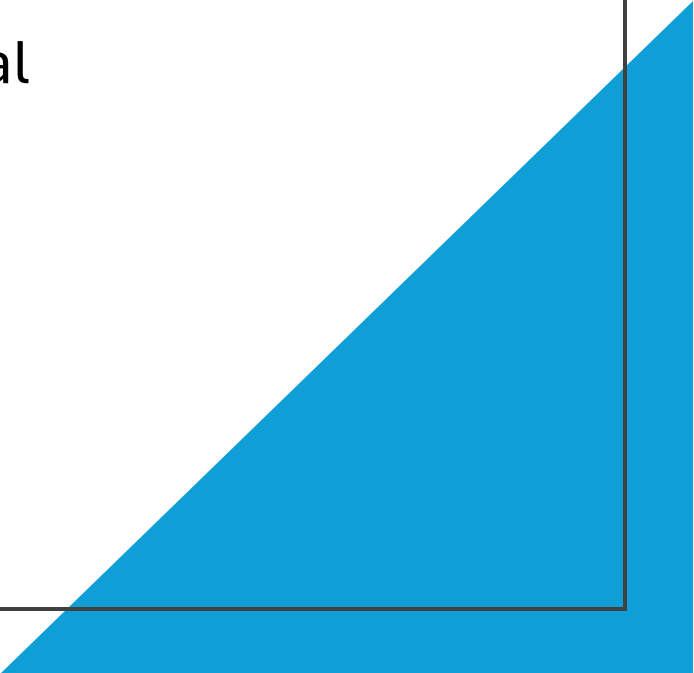
The PORTRAITRON 3000

Final Project Presentation

Avner Fivelovich, Shira Shmueli, Hila Regev Shabath



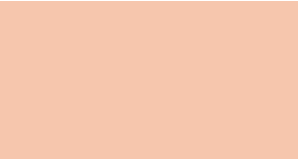
The project combined several different areas:

- Robotics and UR5e control
 - Integration between the digital sketch and the physical drawing process
 - Image processing and sketch generation
 - Mechanical design and 3D printing
 - Tool changing and paper handling
- 

The Original Plan

The original idea was to create a system that could turn a person's photo into a physical portrait drawn by the robot:

1. **Capture & Process**
 - Take a photo of the person and convert it into drawable strokes using **SwiftSketch**.
2. **Draw**
 - Use **UR-RTDE** to control the robot, pick up the marker, and draw the generated strokes on paper
3. **Finish**
 - Stamp the portrait with a commemorative stamp and, if possible, hand the finished drawing to the user



What Changed From the Original Plan?

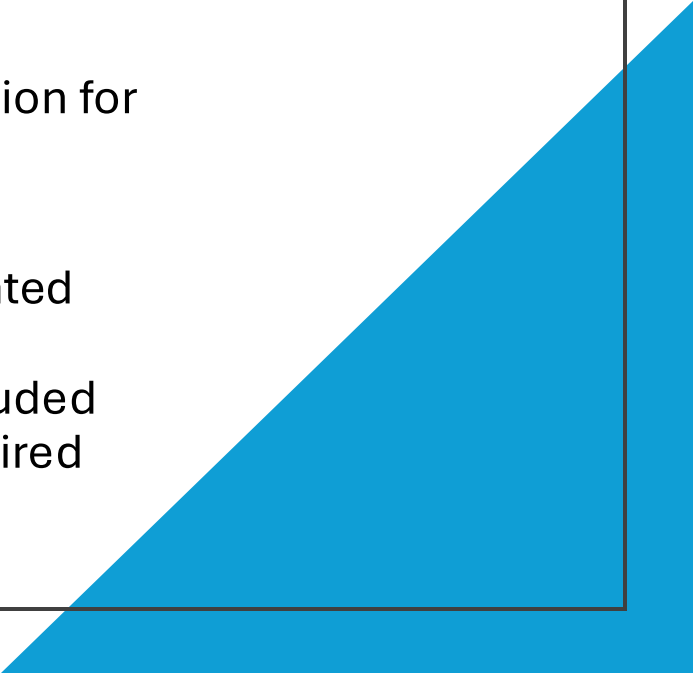
- **Image Processing — From SwiftSketch to Our Own Pipeline**

- **Original plan –**

- Our original plan was to use **SwiftSketch** as an existing solution for converting a photograph into drawable strokes.

- **The problem -**

- When we started testing it on faces, we found that the generated sketches (done in 32 strokes) did not capture facial features accurately enough for reliable portrait recognition. We concluded that to obtain a good result a larger number of strokes is required



- **What we did instead –**

- Estimate the number of strokes needed to obtain an adequate sketch
 - We used ControlSketch to generate images at 64, 96, 128
 - We determined 96 gave a good tradeoff between sketch quality and drawing speed
- Use ControlSketch to produce a source of truth dataset of 96 stroke sketches
- Train our own model based on the SwiftSketch source code, adapting it to output 96 stroke SVGs, as well as train on mac rather than the slurm server, due to slurm server's restrictions.

64



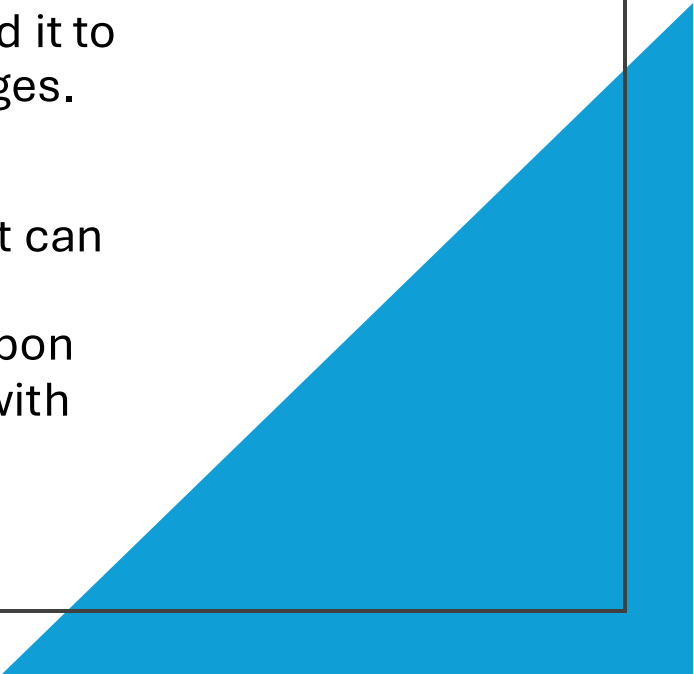
96

128



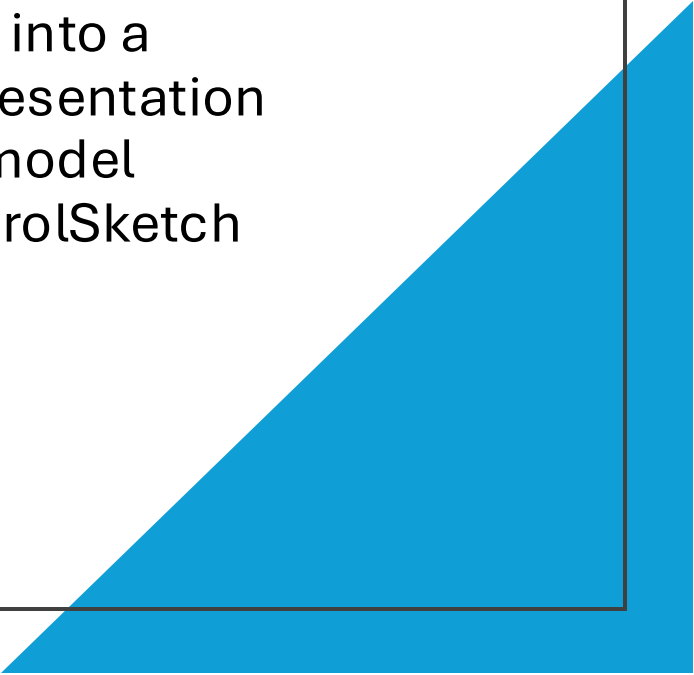


What Changed From the Original Plan?

- **Automating paper handling and handoff**
 - **The problem** – Our original plan was to grab the paper and hand it to the subject, but this proved to be a challenge with single A4 pages.
 - **What we did instead –**
 - We came up with an implementation that uses a paper roll that can be cut by the robot. The cutting process proved to be its own challenge, and we instead opted to have a human cut the roll upon completion of the drawing step, allowing the robot to proceed with grabbing and handing the result.
- 

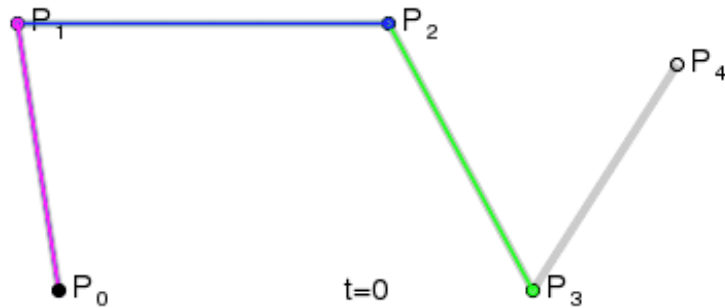


The Sketch Generation Pipeline



- **Input –**
Using a dedicated server the system receives a photograph of the user that he uploaded.
- **Sketch Generation –**
The image is converted into a simplified drawing representation using the SwiftSketch model (with a fallback to ControlSketch optimization).

Converting the Sketch to Robot Movements



- The generated sketch is saved as an **SVG file**. Unlike a regular image made of pixels, SVG represents the drawing using **mathematical geometry** of lines and curves.
- Because the format is well defined, we could directly process these geometric objects.
- In `svg_drawing.py`, each SVG stroke is converted into a sequence of points that the robot can follow using the Bézier curve math (from what we looked it is mostly 3 dimensional curve).

Generated Sketch → SVG → Geometry → Points → Robot Trajectory



Optimizing the Paths

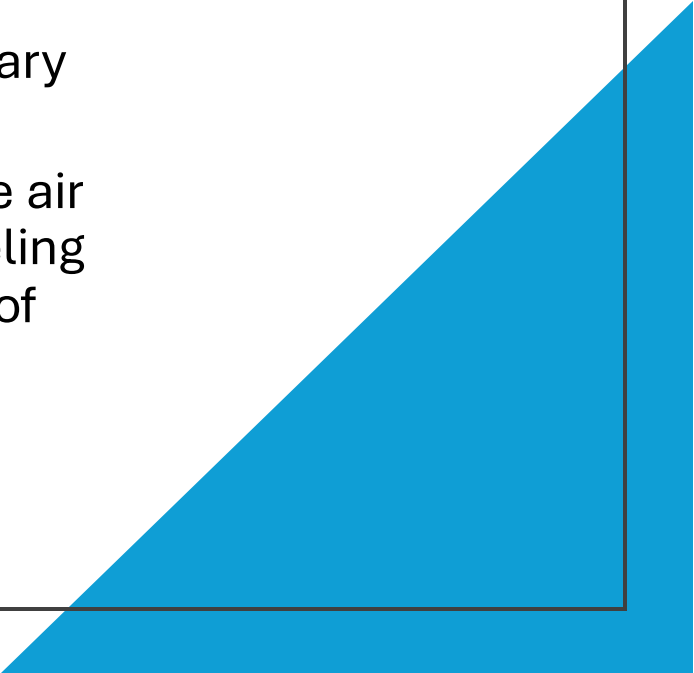
- After generating the list of strokes, we performed several optimization steps to prepare it for execution by the robot:
 - Removing noisy elements using the input face as mask.
 - Connecting lines that have close edges to reduce unnecessary pen lifts.
 - Optimizing the order and direction of strokes to minimize the air distance using a Double-Ended Nearest Neighbor as a Traveling Salesperson Problem (TSP) heuristic. This produces the list of strokes we need to draw sorted by the drawing order
- 

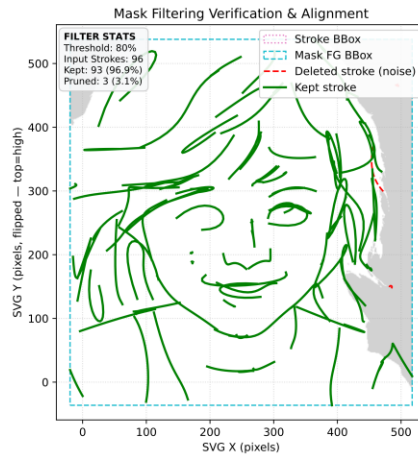
Image Input



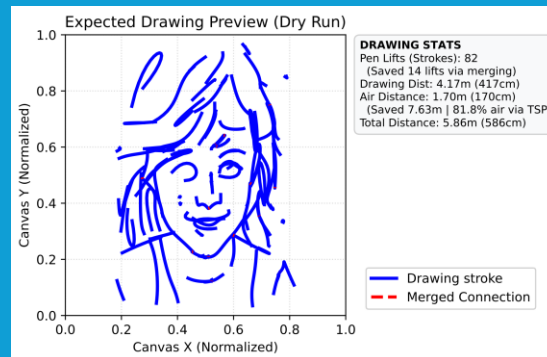
Sketch Generation



Noise Cleaning



Path Optimization

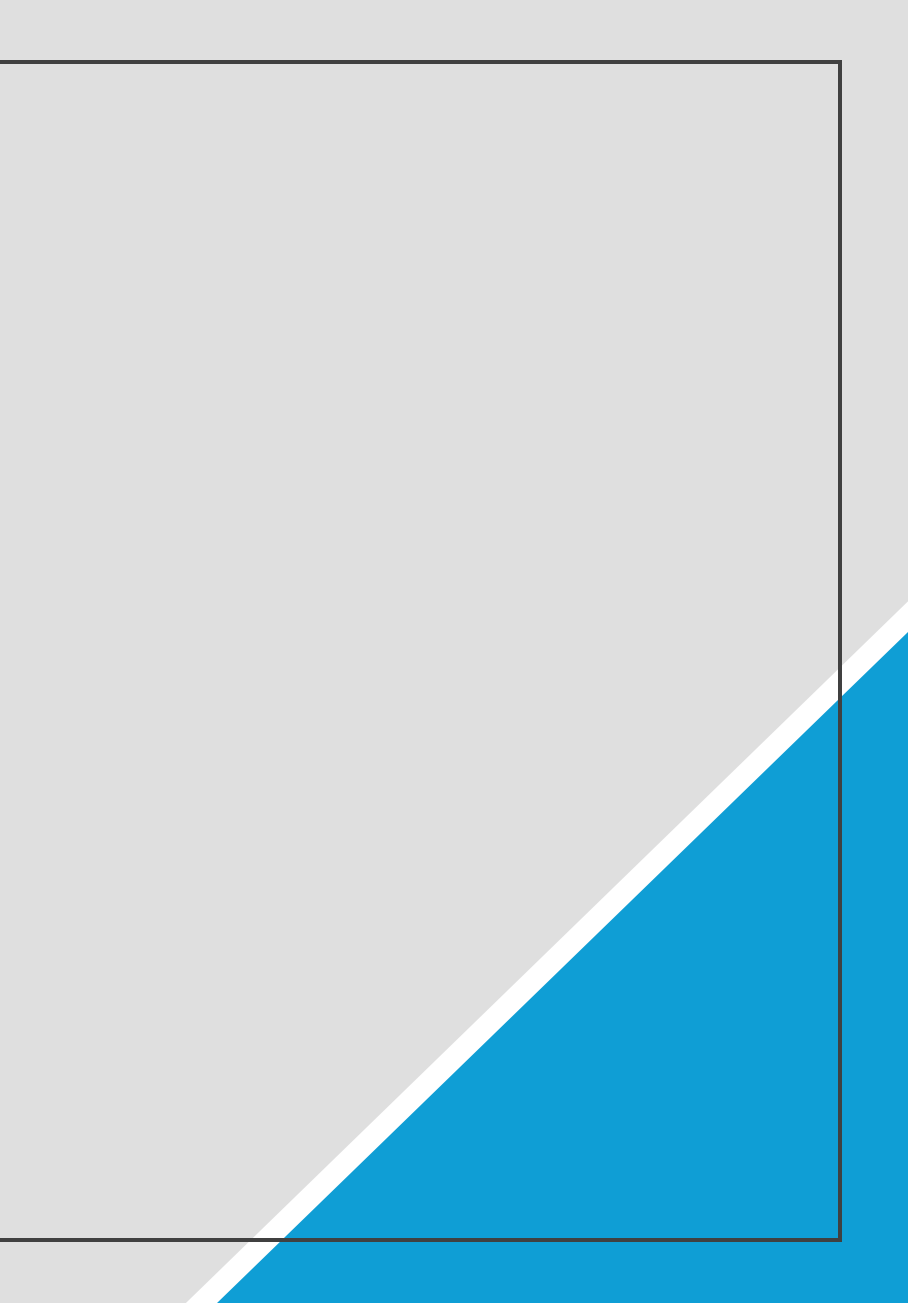


Main Robotics Challenge

- Drawing on the Wall

- One of our main robotics challenges was maintaining the right amount of pressure between the marker and the drawing surface
- The wall was not perfectly straight, so the robot had to continuously adjust while drawing
 - **Too little force:**
The marker loses contact with the paper, causing missing or broken lines
 - **Too much force:**
 - The marker can bend or even break



- 
- 
- So the challenge was to keep enough pressure to draw continuously, without applying too much force
 - We eventually managed this using the Robot's built-in **Force Mode**. Before executing each stroke, we position the marker 5mm above the drawing plane and slowly approach the paper, while checking force values. After receiving **several consecutive high readings** we assume contact and start drawing, while keeping force mode active



Robotics Challenge — Changing the Setup

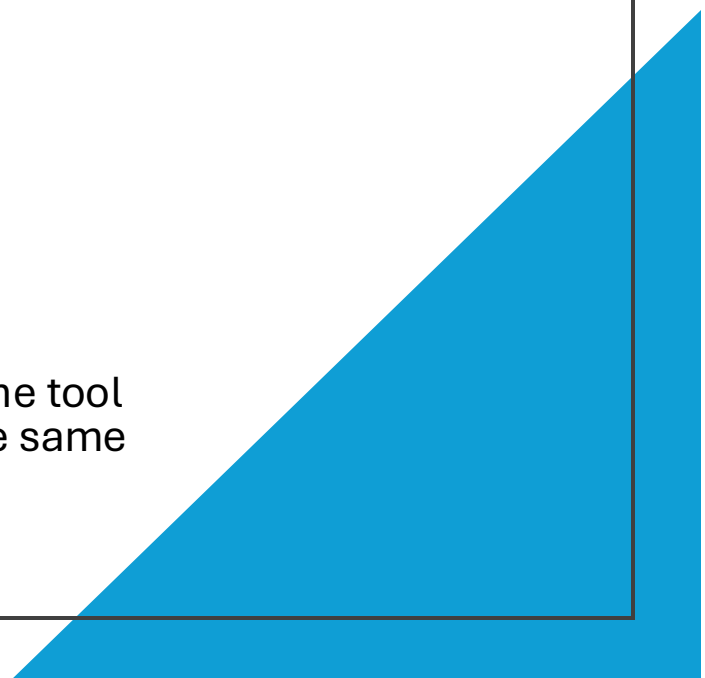
- During the project, we received a new robotic arm with a new operating system (which was reverted later by the staff), changed our setup and moved it to a new location in the lab.
 - Because some of our implementation depended on fixed calibration, we had to update and recalibrate:
 - The drawing position
 - The paper position and size
- 

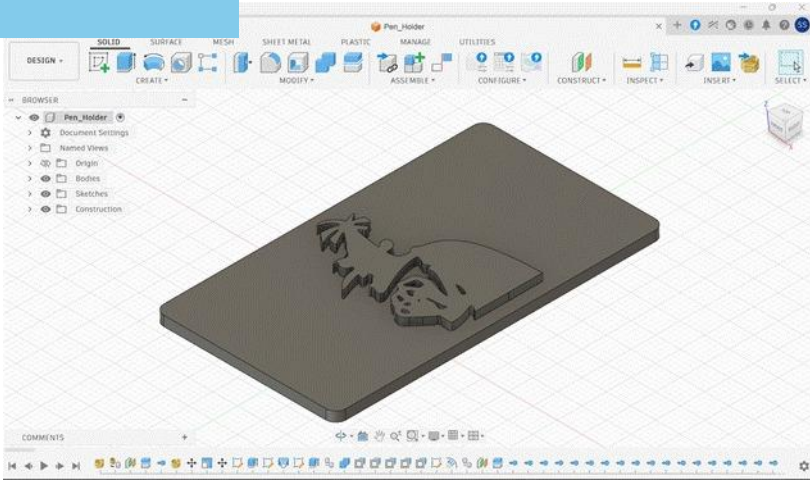


Robotics Challenge — Reliable Tool Changing

- the portraitron needs to switch between several tools during one run:

Magnets → Marker → Stamp → Cutting Tool → magnets

- The challenge was making these tool changes **reliable and repeatable**
 - For every tool, the robot had to:
 - Move to the exact pickup position
 - Connect to the tool correctly
 - Use it
 - Return it to the correct location
 - Even a small error in position or orientation could cause the robot to miss the tool or fail to use it properly. so we had to make sure the robot could perform the same pickup and return process reliably every time
- 



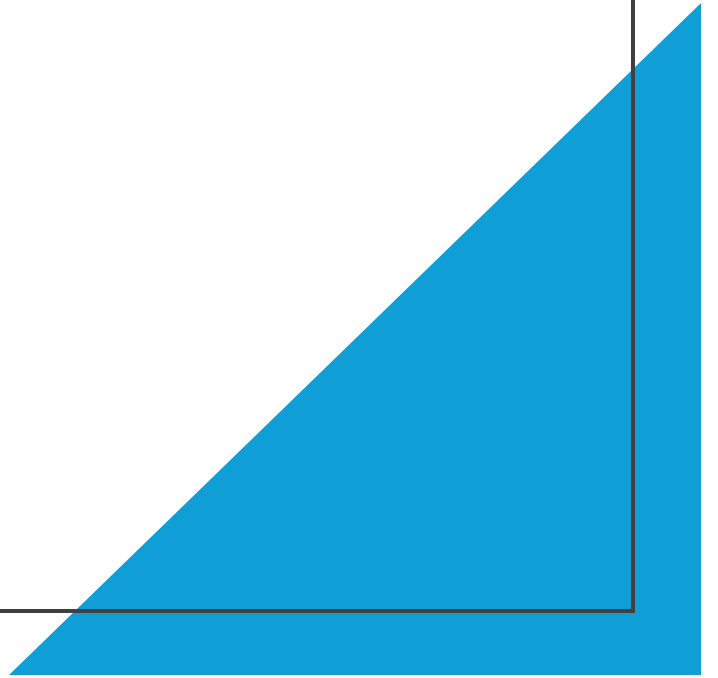
Functionality We Removed or Changed

- **Original face stamp**

The original design included a commemorative stamp containing the team members' faces. We later changed this to a stamp containing text, but the thin lines proved hard to stamp clearly. We ultimately decided to replace the design with an image of our idol and role model:

Functionality We Added

- Compared with the original proposal, the final Portraitron includes several major additions:
 - A web-based user interface.
 - Portrait preview before physical drawing.
 - A paper roll rather than manually inserted individual sheets.
 - Magnetic paper holders to hold the paper roll
 - Automatic(?) portrait cutting.
 - Additional tool-changing operations.
 - A different custom image/sketch-processing pipeline.



The Final Product — User Side

1. **Access –**

The user scans a QR code and opens the Portraitron website.

2. **Take a photo –**

The user takes a photograph of themselves.

3. **Generate a sketch –**

The system processes the photograph and creates a preview of the portrait that the robot will draw.

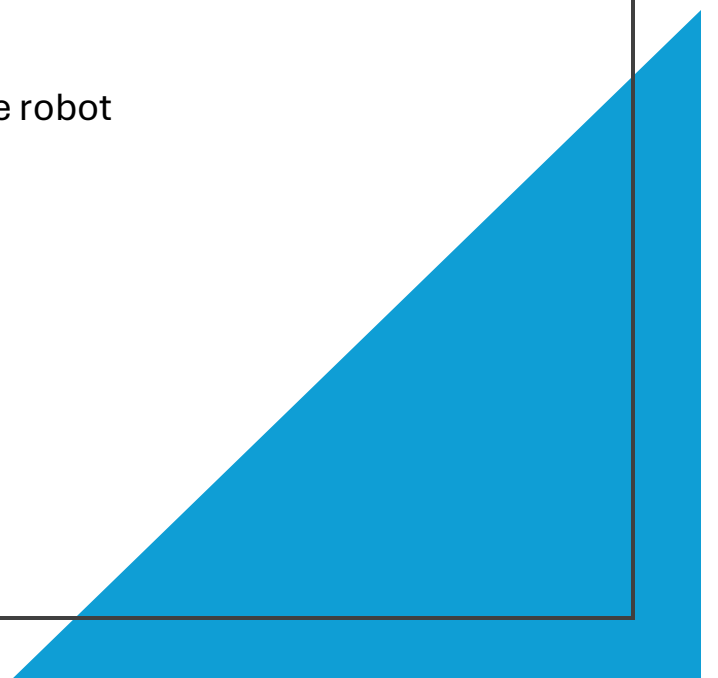
4. **Preview –**

The user can see the generated sketch **before committing to a physical drawing**.

5. **Approve –**

Once the user approves the preview, the drawing data is sent to Portraitron.

Scan → Take Photo → Preview → Approve → Watch Portraitron Draw

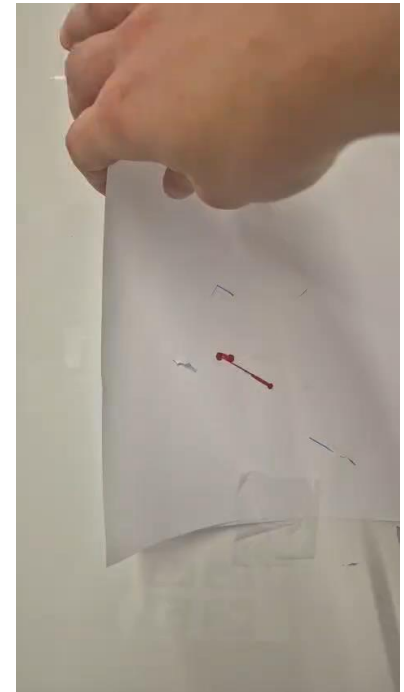
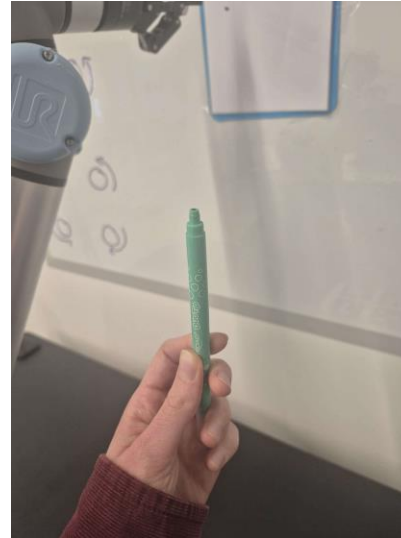


The Final Product — Robot Side



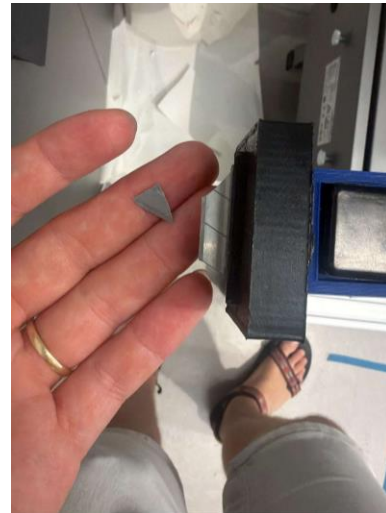
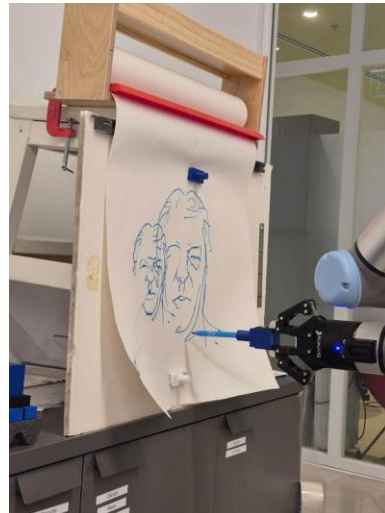
The Development Process

- The first pen mount did not fit the drawing tool
- We initially misunderstood how to use Force Mode effectively



The Development Process

- We redesigned the paper system
- We changed the image-generation pipeline after discovering the limitations of SwiftSketch
- Cutting required a lot of trial and error



Commercial Potential

- the Portraitron can be used as a fun interactive booth for events, similar to a photo booth or event photographer — but instead of taking a photo, a robot draws your portrait.
- Potential locations:
 - Weddings
 - University events
 - Shopping malls
 - Theme parks
 - Tourist attractions

